

Software System Design (2024) Architecture

Assignment 1

软件学院

211850016

李薛成

2024 年 5 月 11 日

1 Task 1 Quality Attribute Scenarios

经过我的研究与对各个质量属性的理解后，我认为在嵌入大语言模型的软件系统中，优先级最高的四个质量属性为：性能 (Performance)、安全性 (Security)、稳定性 (Stability) 和可维护性 (Maintainability)。下面我将分别对这四个质量属性进行一般场景描述和典型具体场景描述。

1.1 性能 (Performance)

性能是指系统处理请求的能力，对于嵌入大语言模型 (LLM) 的软件系统，性能尤为关键，因为它直接影响用户体验和系统的实用性，这通常涉及到模型处理请求和生成响应的速度、系统吞吐量和资源使用效率。

1.1.1 一般场景

Portion of Scenario	Possible Values
Source	用户或自动化系统
Stimulus	请求大语言模型处理大量数据
Artifact	嵌入大语言模型的系统
Environment	正常运行环境、高负载环境、低负载环境
Response	系统处理请求并返回结果
Response Measure	1. 处理时间不超过预定秒数 2. 系统资源占用率维持在一定水平 3. 用户体验满意度较高

1.1.2 典型具体场景

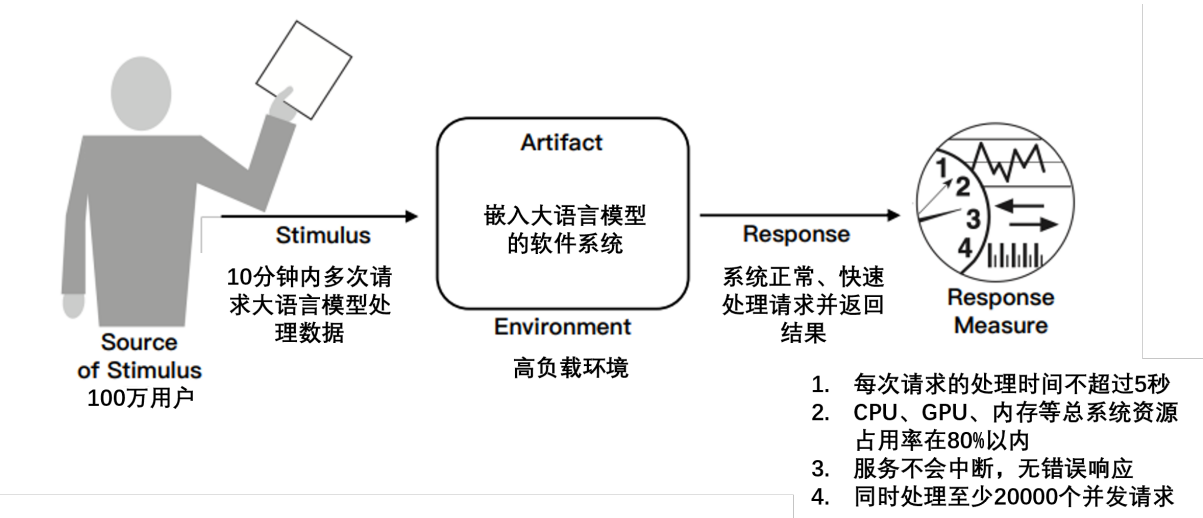


图 1: 性能 (Performance) 典型具体场景

1.2 安全性 (Security)

安全性是保护系统免受未经授权访问和潜在威胁的能力。对于嵌入大语言模型的软件系统，数据处理可能涉及敏感信息、违法信息、违反道德的信息，同时大语言模型很可能成为恶意用户的目标，因此系统必须能够检测和阻止恶意行为，如注入恶意代码、越狱模型安全限制等。因此确保数据完整性、数据不违反系统条例和保护用户隐私是至关重要的。

1.2.1 一般场景

Portion of Scenario	Possible Values
Source	未授权用户或系统
Stimulus	尝试调取违反系统条例的数据或篡改数据
Artifact	大语言模型系统及其存储的数据
Environment	互联网连接环境
Response	检测、阻止违反系统条例的数据输出并报告安全事件
Response Measure	1. 非授权请求尝试被立即检测 2. 系统保护所有敏感数据不被泄露 3. 安全事件记录并报告给管理员

1.2.2 典型具体场景

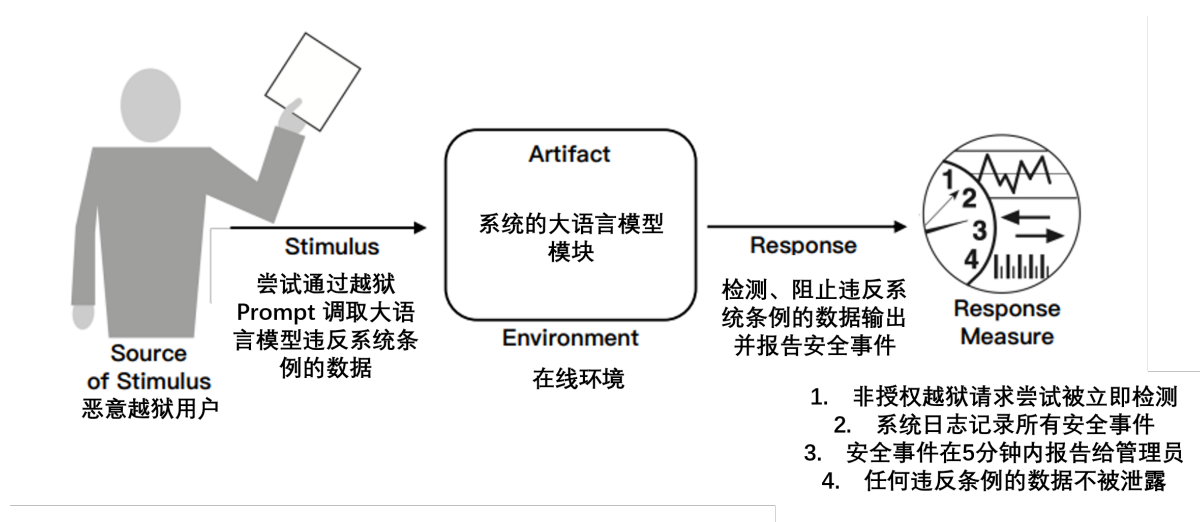


图 2: 安全性 (Security) 典型具体场景

1.3 稳定性 (Stability)

稳定性是指系统在各种操作条件下保持其功能和性能的能力。对于嵌入大语言模型的软件系统来说，保持操作的一致性和预测性至关重要，特别是在数据输入和模型行为可能变化的情

况下。对于异常请求，在返回对应的错误码的同时，系统应该能够保持稳定，不会崩溃或产生不可预测的行为，确保系统的可靠性和可用性。

1.3.1 一般场景

Portion of Scenario	Possible Values
Source	大语言模型、系统组件或外部服务
Stimulus	接收到非标准或意外的输入数据
Artifact	大语言模型、系统及其接口
Environment	正常运行环境
Response	系统处理数据并保持正常运行
Response Measure	1. 系统不崩溃 2. 对非标准输入的正确处理率达到一定水平 3. 系统响应时间不显著增加

1.3.2 典型具体场景

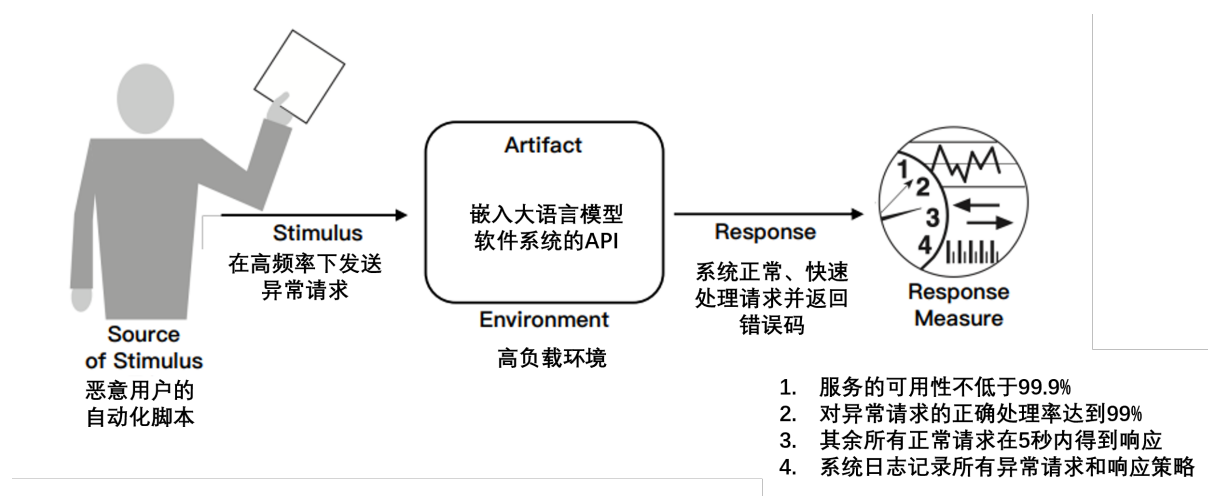


图 3: 稳定性 (Stability) 典型具体场景

1.4 可维护性 (Maintainability)

可维护性是指在软件生命周期中修改系统（包括修复、更新、改进或适应环境变化）的容易程度。对于嵌入大语言模型的软件系统，高可维护性可以减少整体运营成本和提高系统的长期稳定性，确保能够快速适应技术进步和市场需求的变化。

1.4.1 一般场景

Portion of Scenario	Possible Values
Source	开发团队或用户反馈
Stimulus	需要更新或修复系统及组件
Artifact	软件代码库与部署系统
Environment	软件开发周期中，开发和运维环境
Response	对系统进行修改、测试并部署更新
Response Measure	1. 系统在修改后立即恢复全功能 2. 系统更新部署时间不超过一定时间 3. 在维护时系统性能不受显著影响

1.4.2 典型具体场景

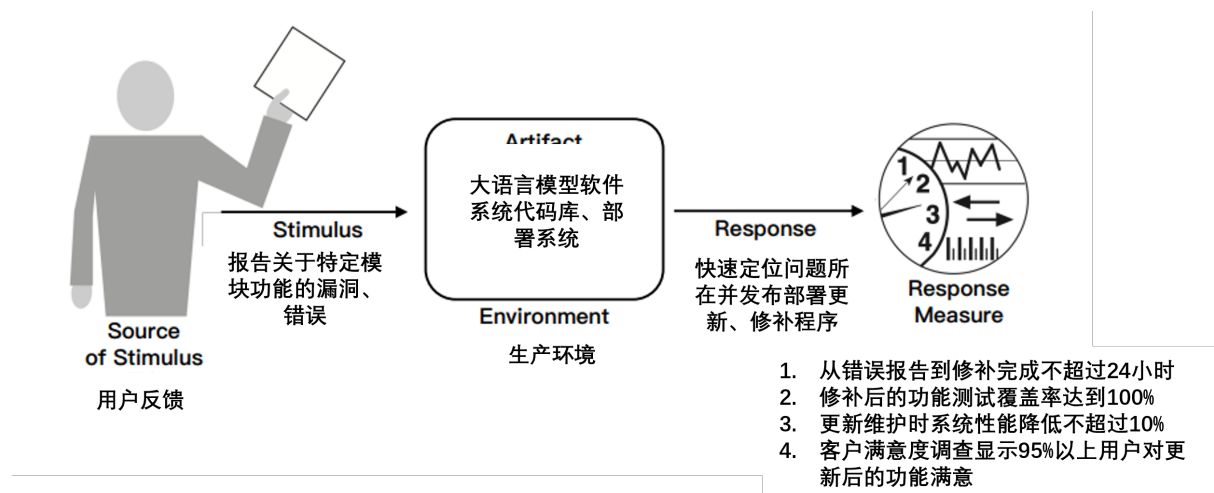


图 4: 可维护性 (Maintainability) 典型具体场景

2 Task 2 Strategies and Tactics

2.1 改进性能 (Performance)

Strategies	Tactics	Performance	Security	Stability	Maintainability
增加资源池, 提高资源利用率	增加服务器数量以处理更多并发请求	显著提升		更多的服务器保障了系统的容错能力	管理复杂度与成本增加, 可能需要更多的维护工作
	使用负载均衡技术分散请求压力, 提高系统吞吐量	显著提升		提升系统承载能力, 避免高负载导致节点崩溃, 增加冗余	管理复杂度增加, 需要定期监控和调整负载均衡策略
优化系统算法	对现有代码进行优化, 减少计算复杂度	显著提升	可能会引入缺陷, 导致安全性问题	减少系统崩溃的可能性	需要持续的代码审查和重构
	采用异步、并行计算技术提高数据处理速度	显著提升	并行计算可能引入并发问题, 导致安全性问题	并行可能导致竞争条件, 增加系统崩溃风险	需要定期监控和调整并行计算策略

2.2 改进安全性 (Security)

Strategies	Tactics	Performance	Security	Stability	Maintainability
加强网络安全管理	采用防火墙对网络流量进行监控和过滤	略有延迟增加	显著提高网络安全, 防止恶意攻击	保障系统的稳定性	需要定期更新和维护防火墙规则
	引入多因素身份验证机制增强访问控制	略有处理延迟	显著提高账户安全性, 防止未授权访问	保障系统的稳定性	需要更新和维护安全策略
加强数据安全	对存储和传输的敏感数据进行加密	增加数据处理时间	显著提高数据安全性		增加代码复杂度, 需要定期更新加密算法
	实现端到端加密保障数据整体安全	增加处理时间	显著提高通信过程中的数据安全性	整体稳定性提升	实施难度高, 需要定期更新安保技术

2.3 改进稳定性 (Stability)

Strategies	Tactics	Performance	Security	Stability	Maintainability
增强错误处理能力	加强系统的错误监控和日志记录功能	略有资源占用增加	提升安全性保障	显著提升错误发现与响应速度	提升系统的可监控性和问题追踪能力
	实现服务的优雅降级策略,当系统负载过高时降低服务质量	略有性能下降	提升安全性保障	显著提升系统不超载运行的能力	系统复杂度增加,需要定期检查和调整降级策略
建立冗余系统	设立备用服务器以确保服务的高可用性	备用服务器可能导致备用系统性能略有下降		显著提升系统的容错能力,保障服务连续性	增加系统维护的工作量和成本
	实施故障转移机制自动切换到备用系统	备用服务器可能导致备用系统性能略有下降		显著增强,保障服务连续性	增加复杂性,维护时可保证服务连续性,需要定期检查

2.4 提升可维护性 (Maintainability)

Strategies	Tactics	Performance	Security	Stability	Maintainability
代码重构	简化和优化现有代码结构		可能会引入缺陷,导致安全性问题		显著提升代码清晰度和可维护性
	引入模块化设计,使功能分区更明确		可能会引入缺陷,导致安全性问题		显著提升系统的可扩展性和模块间的独立性
文档更新	持续更新和完善开发文档			详细的开发文档有助于系统的稳定性	显著增强团队成员的理解和协同工作能力
	提供详尽的 API 文档和使用手册		详细的 API 文档使得系统更加安全	详细的 API 文档有助于系统的稳定性	帮助开发者和用户更好地理解系统功能和操作方法

3 Task 3: Alternative Tactic Debate

3.1 主动冗余 (Active Redundancy)

1. 所有冗余组件均以并行方式响应事件-所有组件均处于相同状态。
2. 仅使用了一个组件的响应，其余组件则被丢弃。
3. 发生故障时，通常不存在停机时间，因为备份是最新的，唯一的切换时间是恢复时间。
4. 确保了系统的高冗余度和容错能力。

3.1.1 好处 (benefits)

1. **高可用性 (High Availability):** 由于备份的主动冗余组件是最新的，因此系统的可用性非常高。
2. **快速恢复 (Fast Recovery):** 当任何一个组件发生故障时，其他正在运行的组件可以立即接管任务，无需任何中断，因此系统的恢复时间非常短。
3. **无需人为干预 (No Human Intervention):** 系统自动管理故障，无需人工介入，提高系统的自动化水平。

3.1.2 成本 (costs)

1. **高资源消耗 (High Resource Consumption):** 主动冗余组件需要消耗更多的资源，包括计算资源、存储资源和网络资源。
2. **高复杂度 (High Complexity):** 系统的设计和实现更加复杂，需要更多的开发、测试和运维工作。

3.2 被动冗余 (Passive Redundancy)

1. 一个组件（主要）响应事件，并通知其他组件（辅助）它们必须进行的状态更新。
2. 发生故障时，系统必须首先确保备份状态足够新，然后才能恢复服务。
3. 关键在于有一个快速而可靠的故障检测和切换机制，确保系统在主组件失败时能够继续运行，尽管这可能涉及到短暂的中断。

3.2.1 好处 (benefits)

1. **资源优化 (Resource Optimization):** 被动冗余组件只有在主组件发生故障时才会启动，因此可以节省资源。
2. **简单易用 (Simple and Easy):** 系统的设计和实现更加简单，易于开发、测试和运维。
3. **高稳定性 (High Stability):** 通过预设的监控和切换机制，确保在主组件失败时可以快速恢复服务。

3.2.2 成本 (costs)

1. **切换延迟 (Switching Delay):** 从主组件切换到备用组件需要时间，可能导致短暂的服务中断。
2. **中资源消耗 (Medium Resource Consumption):** 被动冗余组件在空闲时需要消耗一定的资源，包括计算资源、存储资源和网络资源。
3. **维护复杂度 (Maintenance Complexity):** 需要定期检查和测试维护备用的被动冗余组件，确保其随时可用。

3.3 备件 (Spare)

1. 备用的备件计算平台配置为替换许多不同的故障组件。
2. 不像被动冗余组件那样可能已经安装在系统中，而是在需要时进行快速更换。
3. 关键是管理备件，需要保证备件的及时可用性和适当的存储条件。

3.3.1 好处 (benefits)

1. **资源节约 (Resource Saving):** 备件只在需要时才会启动，因此可以节省资源。
2. **简单易用 (Simple and Easy):** 系统的设计和实现更加简单，易于开发、测试和运维。
3. **高可扩展性 (High Scalability):** 可以根据需要随时添加更多的备件，不需要一开始就投入大量资源。

3.3.2 成本 (costs)

1. **高切换延迟 (High Switching Delay):** 替换失败的部件需要时间，这期间系统可能无法提供服务。
2. **低资源消耗 (Low Resource Consumption):** 备件在空闲时不消耗资源，但需要适当的存储条件。
3. **存储与维护成本 (Storage and Maintenance Costs):** 需要适当的存储和管理体系来确保备件的可用性和有效性。

3.4 适用场景 (Applicable Scenarios)

3.4.1 主动冗余的适用场景

3.4.1.1 航空交通控制系统

- **场景描述:** 航空交通控制系统需要实时处理大量的飞行数据和通信，任何中断都可能导致严重的安全事故。
- **选择原因:** 主动冗余通过多个系统同时运行，可以确保即使一个系统发生故障，其他系统仍能继续无缝运行，从而保证飞行安全和空中交通的流畅。

3.4.1.2 在线支付系统

- **场景描述:** 在线支付系统要求高度的数据一致性和实时处理能力，任何服务中断都可能导致交易失败或双重支付等问题。
- **选择原因:** 主动冗余可以通过多个服务器或数据库同时处理同一任务来确保服务的连续性，减少或消除系统故障对用户的影响。

3.4.2 被动冗余的适用场景

3.4.2.1 数据备份中心

- **场景描述:** 数据备份中心存储大量重要数据，虽然对实时访问要求不高，但必须确保数据的长期完整性和在主数据中心故障时的可用性。
- **选择原因:** 被动冗余通过在主数据中心故障时启动备用系统，可以保证数据的安全和业务的持续性，同时节约在非故障期间的资源消耗。

3.4.2.2 学术研究服务器

- **场景描述:** 学术研究中使用的高性能计算服务器，通常在处理大规模数据或复杂模拟时非常依赖服务器性能。

- **选择原因：**被动冗余允许研究人员在主服务器故障时迅速切换到备用服务器，虽然可能有短暂的中断，但可以在有限的预算内最大化资源的使用效率。

3.4.3 备件的适用场景

3.4.3.1 小型制造业

- **场景描述：**小型制造企业使用的生产设备可能不是持续运行，但设备故障会严重影响生产进度和成本。
- **选择原因：**备件策略允许企业仅在设备发生故障时替换所需部件，有助于控制成本，同时保持生产线的最小中断。

3.4.3.2 学校 IT 设施

- **场景描述：**学校的 IT 设施通常包括大量的电脑和相关设备，这些设备的全面冗余成本过高，但需要保证基本的教学活动不受影响。
- **选择原因：**备件策略可以在经济有效的前提下，确保教学所需的 IT 设施在发生故障时能够得到快速修复，减少教学活动的中断。

3.5 决策标准 (Decision Criteria)

选择适当的可用性战术应基于以下标准：

- **关键性：**系统的关键性越高，越倾向于使用主动冗余。
- **成本限制：**预算限制较大时，应考虑被动冗余或备件策略。
- **服务中断的可接受性：**如果业务可以接受短暂中断，被动冗余或备件可能更合适。
- **资源可用性：**如果有充足的资源（如资金、硬件和能源），则可以考虑主动冗余；否则，应考虑更节省资源的选项。
- **系统复杂度和维护能力：**如果组织有能力维护复杂的系统，则可采用主动冗余；如果维护能力有限，可能需要选择较简单的被动冗余或备件策略。

综合考虑上述因素，可以选择最适合特定场景和需求的可用性战术。正确的选择不仅能保证系统的高效运行，还能在成本和性能之间找到最佳平衡。